



CorpusMind Voice is the audio companion of **CorpusMind**, the local-first research environment for corpus linguistics and multimodal discourse analysis. It converts recordings (MP3, MP4, M4A, AAC, OGG, OPUS, WAV, FLAC and more) or live microphone input into a linguistically annotated corpus on your own machine. No account is needed, nothing is uploaded, and no cloud API is ever contacted: the privacy model is absolute by design.

1 Installation

GET RUNNING

WEB & PWA

Runs anywhere Node.js (or Bun) runs: `bun install`, `bun run db:push`, then `bun run dev` and open `http://localhost:3000`. Installable as a **PWA** from the browser address bar, so a phone or tablet can **record and analyse on the go**, entirely on-device. A first-launch **Welcome tour** (three short pages, English/Arabic) introduces the app and the privacy model, and a **Light / Dark / System** theme toggle lives in the header.

DESKTOP & PYTHON WORKER

Native builds for **Windows (.exe / .msi)**, **macOS (Apple Silicon & Intel .dmg)** and **Linux (.deb)** ship with each GitHub release and need no Python. For research-grade output add the Python worker (`pip install -r python/requirements.txt`); optional **Montreal Forced Aligner** adds phoneme-grade boundaries (`arabic_mfa` / `english_us_arpa` models and dictionaries).

2 Models, packages and hardware

BEFORE YOUR FIRST JOB

Open **Settings and Diagnostics** before your first job. Models persist in the app's private data folder, survive restarts and updates, are reused offline, and can be deleted at any time to reclaim space.

tiny 75 MB **base** 145 MB **small** 484 MB **medium** 1.5 GB **large-v3** 3.1 GB · recommended

- Hardware & Diagnostics** reads your CPU, RAM and NVIDIA GPU (via `nvidia-smi`). With CUDA, ASR runs INT8-float16 on the GPU; without it the app warns and falls back to CPU (INT8), about 2 × real time. Pin the device per job (Auto / CPU / GPU).
- Local LLM**: Ollama (port 11434) and LM Studio (port 1234) are detected automatically, like the parent CorpusMind; on desktop the app can start `ollama serve` for you.
- Filler lexicon new in v1.2.0**: the hesitation markers tagged in stage 5 are editable per language (English and Arabic); adapt them to your dialect, for example add Gulf or Levantine markers. Changes apply to newly processed recordings.

3 Record and process: the six steps

STUDIO

1. In **Studio**, drop a file or record from the microphone (*Stop recording* uploads the take; a webm/opus → mp4/aac → ogg fallback keeps mobile recording working). 2. Pick the **language** (English; Arabic, vernacular عامية; Arabic, MSA فصحي), the **compute device** and the **Whisper model**. 3. Click **Start pipeline**; progress streams live per stage.

#	STAGE	WHAT HAPPENS
1	Ingest & preprocess	container decoded to 16 kHz mono PCM WAV (PyAV / pydub)
2	ASR	faster-whisper (chosen model) INT8; word timestamps + confidence
3	Forced alignment	MFA refines word and phoneme boundaries (when installed)
4	Prosody	Praat via Parselmouth: F0 (50–400 Hz), intensity, jitter, shimmer, HNR
5	Disfluency	pauses > 200 ms, fillers (أيه, آه, يعني), repeats, false starts, interruptions, lengthenings
6	Structured output	SQLite (3 tables) + JSON + CSV + TEI/XML + TextGrid + EAF + SRT/VTT, written locally

No Python worker installed? The app runs its **simulation engine** instead (same six stages, demo corpus) and labels the job *Simulation*, so demo data is never mistaken for real transcriptions.

4 Correct the transcript

CONFIDENCE-CODED EDITOR

In the **Transcript Editor**, every token carries its ASR confidence as a colour: click a token to correct it, and saving marks it *edited* and queues a **single-utterance re-alignment pass**, so a one-word fix never re-processes the file. Utterance text rebuilds from its tokens, so exports always match what you hear.

● confidence ≥ 0.85 ● 0.60–0.85, review ● < 0.60, fix first ▶ play any utterance from the recording

Click the **speaker label** to relabel speakers in multi-speaker recordings. Each utterance lists its disfluencies and prosody summary (F0, intensity, jitter, shimmer, HNR); a **Save to device** row writes JSON / CSV / TEI/XML / TextGrid / EAF / SRT / WebVTT / SQLite.

The **Linguistic Analysis** tab computes a full academic report locally; every table exports to CSV, the whole report to JSON.

Overview

tokens, types, TTR, speech rate (wpm), utterances, hapax legomena, confidence distribution.

Frequency

counts, per-1,000 rates, Gries **DP dispersion**, function-word filter, bar chart, log-log **Zipf curve**.

KWIC

concordancer with context window, timestamps, audio playback per hit; **normalized** (Arabic-aware), **whole-word** and **regex** modes with a live hit count.

Keywords

log-likelihood **G²**, **LogRatio** and **%DIFF** effect sizes (Hardie 2014) against your other sessions, or a built-in function-word reference.

Collocations & N-grams

MI, **t-score**, **logDice**, top n-gram lists, and a **node-word collocates explorer** (span 1–5, sortable).

Lexical diversity

length-robust **MATTR** (window 50) and **MTLD** (≥ 0.72), plus plain TTR and lexical density.

Readability

Flesch Reading Ease and Flesch–Kincaid grade (English), language-neutral **LIX** and long-word share.

Speech & Prosody

disfluency rates per 1,000 tokens by type, pause statistics, Praat aggregates (F0 mean/range, intensity, jitter, shimmer, HNR).

6 The corpus across sessions

NEW IN V1.2.0

The **Corpus** tab aggregates everything processed so far. The **Sessions** table lists each recording with language, duration, utterance and token counts, and loads it back into the editor and analysis tabs. The **Frequency** view computes the corpus word list with **DP dispersion measured across sessions** (0 = evenly spread, 1 = concentrated in one session), and the **Concordance** view runs one normalized / whole-word / regex search across every session, with every hit playable from the recording.

7 Corpus metadata (TEI header)

INTEROPERABILITY

In **Metadata**, describe the session: title, speaker name or pseudonym, dialect, gender, age, recording date and place, genre, licence and free notes. These fields are embedded into the `<teiHeader>` of every saved file, exactly the fields CorpusMind expects when importing. Arabic metadata is fully supported, and the **Launch CorpusMind** button opens the companion app with this corpus ready to query.

8 Local AI assistant (optional)

STAYS ON YOUR MACHINE

The **Assistant** tab chats with a *local* model through **Ollama** (`http://127.0.0.1:11434`) or **LM Studio** (`http://127.0.0.1:1234`), for example “list utterances with more than two fillers”. Start Ollama with `ollama serve`, pull a model (`ollama run llama3.1`) or load one in LM Studio, then pick it in the tab or in Settings. Three per-transcript tools run through the same local model: **Summarise**, **Preview without fillers** and **Suggest topic tags**. If no provider is reachable, the panel says so and never substitutes a cloud service.

9 Troubleshooting

QUICK FIXES

SYMPTOM	FIX
Job labelled <i>Simulation</i>	Python worker missing → <code>pip install -r python/requirements.txt</code>
<i>MFA skipped</i> in stage 3	optional: install MFA + dictionaries (see section 1)
<i>No NVIDIA GPU detected</i>	expected on CPU-only machines; INT8 CPU mode is used
Whisper model missing in Studio	Settings and Diagnostics → Models & packages → Download
Microphone button does nothing	grant mic permission in the browser/OS, or upload a file; recording needs HTTPS or localhost
Assistant unreachable	start <code>ollama serve</code> or load a model in LM Studio (port 1234), then re-open the tab
Downloaded model “vanished”	it cannot: models persist in the app data folder; check the path shown in Settings